SEGURANÇA DA INFORMAÇÃO

Rainbow Tables com Diceware

Implementação em Python, validação e análise experimental

Autores: A preencher pelos alunos

Referência didática: Ivan Sendin, FACOM/UFU. Aula de 13 de outubro de 2026 [1].

Ambiente das medições: **Ambiente remoto de execução do ChatGPT (contêiner Linux)**.

Este documento reúne implementação, testes e resultados. Os tempos e recursos descritos pertencem ao ambiente declarado, não necessariamente ao computador dos alunos. Os nomes dos autores devem ser preenchidos antes da entrega.

Resumo

Implementou-se uma Rainbow Table para senhas formadas por uma ou duas entradas da lista Diceware original, com SHA-256 e o prefixo fixo GBC083. Foram avaliadas seis configurações, com 3 repetições e 200 consultas aleatórias por execução. A correção foi verificada em universos pequenos por busca exaustiva e nos experimentos por comparação com a cobertura exata das cadeias. As medições distinguem tempo de construção, armazenamento, alocações Python e tempo de consulta.

Na configuração de 500 cadeias e 20 passos, a cobertura média para uma palavra foi 63.77%. Com duas palavras, 10.000 cadeias e 20 passos cobriram 0.3300% do universo. O crescimento do espaço de senhas explica a grande diferença, mesmo com aumento do orçamento de construção.

Organização: modelo e algoritmo; metodologia; resultados; análise e reprodução; referências; código-fonte integral dos módulos de execução e testes.

1. Modelo e implementação

A lista contém D = 7.776 entradas [2]. Cada senha possui exatamente k palavras, com repetição permitida, ordem relevante e um espaço ASCII como separador. Não há conversão de caixa nem remoção de pontuação. Os códigos de cinco dados identificam as entradas e não compõem a senha.

k	Espaço N = D^k	Orçamento m × t
1	7.776	10.000 passos
2	60.466.176	200.000 passos

O hash é H(s) = SHA256(b"GBC083" + UTF8(s)), conforme [1]. O retorno interno tem 32 bytes. O vetor H("1234") é verificado por teste automatizado. O prefixo público e fixo não equivale a um salt aleatório individual: a mesma RT continua reutilizável para todas as senhas submetidas a esse esquema.

A redução da coluna i é o inteiro big-endian de SHA256(b"RT-R1" + i em 8 bytes big-endian + h), módulo N. O resultado é um ID decodificado em k dígitos na base D, que selecionam as palavras. Assim, qualquer saída corresponde a uma senha válida. A segunda aplicação de SHA-256 mistura a cor com o hash; seu custo integra os tempos medidos. O módulo introduz um viés desprezível neste espaço frente a 2^256, sem promessa de ausência de colisões.

Cada cadeia tem t passos: x[j+1] = R[j](H(x[j])), de j = 0 até t-1. Somente (x[0], x[t]) é armazenado. Os inícios são amostrados sem reposição. Um dicionário Python associa cada final a uma lista de inícios, preservando cadeias com finais iguais. A lista Diceware e metadados acompanham a serialização JSON.

A consulta testa posições do hash de t-1 até 0. Em cada hipótese, aplica a cor da posição e segue até o fim. Quando o final existe, reconstrói todas as cadeias candidatas ainda não examinadas e compara os hashes. Somente uma igualdade de hash produz resposta. O último ID x[t] não conta na cobertura daquela cadeia, pois não foi hasheado nela.

Construção: O(m × t) passos; extremos: O(m), além da lista. A consulta de hipóteses pode exigir O(t²) passos, acrescidos de O(c × t) para c cadeias candidatas distintas. Cores reduzem fusões causadas por encontros em colunas diferentes; encontros na mesma coluna ainda compartilham o restante da cadeia.

2. Metodologia e ambiente

Item	Valor
CPU	AMD EPYC 9V74 80-Core Processor
CPUs lógicas visíveis	9
RAM visível	9.73 GiB
Sistema	Linux-6.18.55-x86_64-with-glibc2.39
Python	3.12.14
Execução	sequencial, um processo; recursos visíveis podem ser do host
Limites cgroup	{'cpu.max': '800000 100000', 'memory.max': '8589934592'}

Quando cgroup é informado, cpu.max usa quota/período em microssegundos e memory.max usa bytes. Valores visíveis podem refletir o host; limites do contêiner são mais restritivos. Não foi utilizado paralelismo ou GPU. Tempos em ambiente compartilhado podem variar por carga externa.

Cada configuração usa sementes 100 até 102 para os inícios. Os alvos usam sementes 900000 + 1000k + repetição, com 200 sorteios uniformes com reposição por execução. Dentro do mesmo k e repetição, as configurações recebem os mesmos alvos. A seleção de alvos é independente da seleção dos inícios. As repetições variam a tabela e a amostra.

Tempos são medidos com perf_counter. Construção exclui carregamento da lista e gravação em disco. Consulta exclui geração do hash alvo e leitura do arquivo: a tabela já está em memória. O tamanho em disco inclui JSON, lista e metadados. Uma reconstrução separada, com tracemalloc, mede o pico de alocações Python da construção; ele não representa RAM/RSS total e não inclui a lista carregada antes da medição.

A cobertura exata é auditada reconstruindo as cadeias e armazenando os IDs hasheados em um conjunto. Essa auditoria consome memória adicional, excluída das métricas da RT. Para cada alvo, exige-se equivalência entre pertencimento ao conjunto e sucesso da busca. Toda senha encontrada também é validada pelo hash.

Sete testes automatizados cobrem: vetor do professor, integridade da lista, codificação bijetiva, redução determinística e válida, busca exaustiva versus cobertura em espaços pequenos, finais duplicados e persistência, ausência e entradas inválidas. O arquivo testes.txt registra a execução inicial aprovada.

3. Resultados medidos

Valores abaixo são médias entre repetições. Sucesso é a média das taxas amostrais; cobertura é a fração exata de IDs distintos hasheados. KiB = 1.024 bytes. A coluna de memória mede alocações Python, não RAM total.

k / m / t	Cobertura %	Sucesso %	Geração s	Busca ms	Disco KiB	Pico KiB
1 / 100 / 100	63.280	59.67	0.014	7.542	56.1	18.5
1 / 200 / 50	64.476	62.17	0.014	2.170	57.1	24.4
1 / 500 / 20	63.769	64.67	0.015	0.348	60.1	67.7
2 / 2000 / 100	0.330	0.00	0.371	8.236	97.3	380.3
2 / 4000 / 50	0.330	0.33	0.333	1.925	139.5	764.2
2 / 10000 / 20	0.330	0.33	0.345	0.329	266.2	1844.6

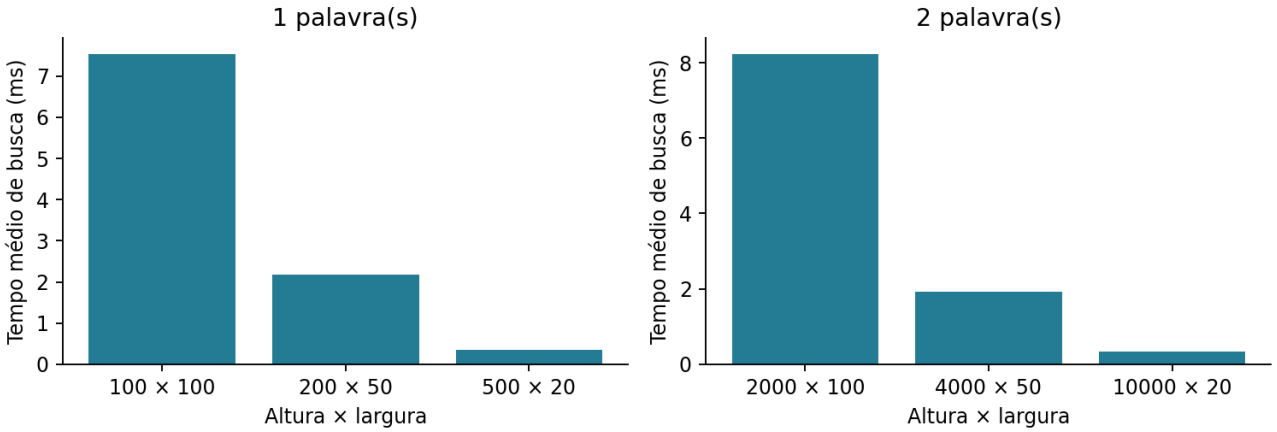


Figura 1. Tempo médio de consulta por configuração, incluindo sucessos e falhas. A escala é própria de cada painel. As configurações de uma e duas palavras têm orçamentos diferentes.

As configurações de cada k mantêm m × t constante. Mesmo assim, a cobertura varia porque os percursos repetem senhas e podem se fundir. O produto m × t não é uma contagem de senhas distintas.

4. Variabilidade e baixa taxa de sucesso

Cada linha apresenta uma execução. O IC95% de Wilson descreve a incerteza binomial da taxa de sucesso para aquela tabela fixa, sob sorteios uniformes independentes. Não é um intervalo para tempos nem um intervalo que resuma todas as tabelas possíveis.

k / m / t	Semente	Acertos	IC95% sucesso	Busca média ms
1 / 100 / 100	100	121/200	53.59% a 67.02%	6.723
1 / 100 / 100	101	118/200	52.08% a 65.58%	7.492
1 / 100 / 100	102	119/200	52.58% a 66.06%	8.411
1 / 200 / 50	100	120/200	53.08% a 66.54%	2.228
1 / 200 / 50	101	129/200	57.65% a 70.80%	2.091
1 / 200 / 50	102	124/200	55.11% a 68.44%	2.191
1 / 500 / 20	100	128/200	57.14% a 70.33%	0.351
1 / 500 / 20	101	131/200	58.68% a 71.74%	0.366
1 / 500 / 20	102	129/200	57.65% a 70.80%	0.327
2 / 2000 / 100	100	0/200	0.00% a 1.88%	8.422
2 / 2000 / 100	101	0/200	0.00% a 1.88%	8.276
2 / 2000 / 100	102	0/200	0.00% a 1.88%	8.010
2 / 4000 / 50	100	1/200	0.09% a 2.78%	1.923
2 / 4000 / 50	101	0/200	0.00% a 1.88%	1.902
2 / 4000 / 50	102	1/200	0.09% a 2.78%	1.951
2 / 10000 / 20	100	1/200	0.09% a 2.78%	0.377
2 / 10000 / 20	101	0/200	0.00% a 1.88%	0.301
2 / 10000 / 20	102	1/200	0.09% a 2.78%	0.308

Com duas palavras, a cobertura fica próxima de 0,33%. Em 200 tentativas, isso representa aproximadamente 0,66 acerto esperado por tabela. Portanto, zero acertos é compatível com uma implementação correta e cobertura não nula. A auditoria exata evita interpretar uma amostra pequena como prova de ausência de cobertura.

5. Análise, limitações e reprodução

Para uma palavra, passar de 100 × 100 para 500 × 20 alterou o tempo médio de consulta de 7.542 ms para 0.348 ms e o arquivo de 56.1 KiB para 60.1 KiB. A diferença evidencia o compromisso entre extremos armazenados e recomputação. A lista incorporada ao JSON gera custo fixo, reduzindo a proporção aparente de crescimento do arquivo.

O orçamento de duas palavras é vinte vezes maior, mas seu universo é 7.776 vezes maior. A redução de sucesso decorre principalmente dessa relação. Os resultados não demonstram fraqueza criptográfica do SHA-256 nem se estendem a senhas longas Diceware. Explora-se um espaço de candidatos pequeno ou parcialmente coberto.

Não foi medido um baseline de força bruta ou dicionário completo: não se afirma superioridade sobre essas alternativas. Para apenas 7.776 entradas, um dicionário completo é simples e pode ser preferível. A RT demonstra a troca tempo-memória; sua pré-computação só pode ser amortizada quando reutilizada em consultas compatíveis.

Limitações: três formatos por espaço; uma família de reduções; poucos alvos para o cenário raro de duas palavras; tempos sujeitos ao ambiente; alocações Python não equivalem a RSS; tabela JSON sem otimização binária. Melhorias possíveis incluem mais repetições, mais consultas para taxas baixas, comparação com força bruta, tabelas binárias e estudo de diferentes famílias de redução. Não se garante cobertura integral.

Reprodução (na pasta do projeto, Python 3.10+):

```
python -m unittest -v
python experimentos.py --output resultados_meu_pc
python -m pip install -r requirements-relatorio.txt
python relatorio.py --input resultados_meu_pc/resultados.json \
  --output relatorio_meu_pc.pdf --ambiente "Computador dos alunos" \
  --autores "Nome 1; Nome 2"
```

Referências e integridade

[1] SENDIN, Ivan. Segurança da Informação: Aula Rainbow Tables. FACOM, Universidade Federal de Uberlândia, 13 out. 2026. Material fornecido pelo usuário, especialmente slides 46 e 47.

[2] REINHOLD, Arnold. Diceware word list. Link da aula: http://world.std.com/~reinhold/diceware.wordlist.asc. Cópia consultada em https://theworld.com/~reinhold/diceware.wordlist.asc, 21 out. 2026. Lista integral no pacote. A assinatura PGP não foi validada.

SHA-256 do arquivo Diceware utilizado:

```
3cd6164a99e95381f8620aec782a933545bcd5833fa331d267a6829f6665256e
```

Os módulos seguintes são transcritos do projeto que acompanha o relatório. O JSON inclui os resultados individuais das consultas. A geração de PDF é utilitária e não faz parte do algoritmo avaliado.

Apêndice - rt.py

Código-fonte; linhas longas quebradas apenas para exibição.

```
"""Rainbow Table didática: SHA256(b'GBC083' + senha UTF-8)."""
import argparse
import hashlib
import json
import random
import re
from pathlib import Path

DEFAULT_LIST = Path(__file__).parent / 'data' / 'diceware.wordlist.asc'


def load_words(path=DEFAULT_LIST):
    rows = []
    for line in Path(path).read_text(encoding='utf-8').splitlines():
        match = re.fullmatch(r'([1-6]{5})\s+(\S+)', line)
        if match:
            rows.append((match[1], match[2]))
    if len(rows) != 7776 or len({x[0] for x in rows}) != 7776:
        raise ValueError('Esperadas 7776 entradas Diceware com códigos únicos.')
    words = [word for _, word in sorted(rows)]
    if len(set(words)) != len(words):
        raise ValueError('Palavras duplicadas na lista.')
    return words


def my_hash(password):
    return hashlib.sha256(b'GBC083' + password.encode('utf-8')).digest()


class Space:
    def __init__(self, words, k=1):
        if not words or len(set(words)) != len(words) or k < 1:
            raise ValueError('Lista única não vazia e k >= 1 são necessários.')
        if any(' ' in word for word in words):
            raise ValueError('As palavras não podem conter espaço.')
        self.words, self.k = list(words), k
        self.n = len(words) ** k

    def password(self, index):
        if not 0 <= index < self.n:
            raise ValueError('Índice fora do espaço.')
        digits = []
        for _ in range(self.k):
            index, digit = divmod(index, len(self.words))
            digits.append(self.words[digit])
        return ' '.join(reversed(digits))

    def reduce(self, digest, column):
        # Mistura determinística com separação de domínio e cor de 8 bytes.
        mixed = hashlib.sha256(b'RT-R1' + column.to_bytes(8, 'big') + digest).digest()
        return int.from_bytes(mixed, 'big') % self.n


class RainbowTable:
    def __init__(self, space, width, endpoints):
        if width < 1:
            raise ValueError('Largura deve ser positiva.')
        self.space, self.width, self.endpoints = space, width, endpoints

    @classmethod
    def build(cls, space, height, width, seed=0):
        if not 1 <= height <= space.n or width < 1:
            raise ValueError('Dimensões inválidas.')
        ends = {}
        for start in random.Random(seed).sample(range(space.n), height):
            current = start
            for column in range(width):
                current = space.reduce(my_hash(space.password(current)), column)
            # Nunca descartar inícios que compartilham um final.
            ends.setdefault(current, []).append(start)
        return cls(space, width, ends)

    def lookup(self, target):
        if len(target) != 32:
            raise ValueError('Hash SHA-256 deve ter 32 bytes.')
```

Apêndice - rt.py

Código-fonte; linhas longas quebradas apenas para exibição.

```
076         checked = set()
077         for position in range(self.width - 1, -1, -1):
078             current = self.space.reduce(target, position)
079             for column in range(position + 1, self.width):
080                 current = self.space.reduce(my_hash(self.space.password(current)), column)
081             for start in self.endpoints.get(current, []):
082                 if start in checked:
083                     continue
084                 checked.add(start)
085                 candidate = start
086                 for column in range(self.width):
087                     password = self.space.password(candidate)
088                     digest = my_hash(password)
089                     if digest == target:
090                         return password
091                     candidate = self.space.reduce(digest, column)
092         return None
093 
094     def coverage(self):
095         # Auditoria separada: inclui somente posições efetivamente hasheadas.
096         seen = set()
097         for starts in self.endpoints.values():
098             for start in starts:
099                 current = start
100                 for column in range(self.width):
101                     seen.add(current)
102                     current = self.space.reduce(my_hash(self.space.password(current)), column)
103         return seen
104 
105     def save(self, path):
106         obj = {'version': 1, 'hash': 'SHA256(GBC083||UTF8)',
107                'reduction': 'SHA256(RT-R1||column_u64be||digest) mod N',
108                'words': self.space.words, 'k': self.space.k,
109                'width': self.width, 'endpoints': self.endpoints}
110         Path(path).write_text(json.dumps(obj, ensure_ascii=False, separators=(',', ':')),
    encoding='utf-8')
111 
112     @classmethod
113     def load(cls, path):
114         obj = json.loads(Path(path).read_text(encoding='utf-8'))
115         if obj['version'] != 1 or obj['hash'] != 'SHA256(GBC083||UTF8)' or obj['reduction'] !=
    'SHA256(RT-R1||column_u64be||digest) mod N':
116             raise ValueError('Formato incompatível.')
117         return cls(Space(obj['words'], obj['k']), obj['width'],
118                    {int(end): starts for end, starts in obj['endpoints'].items()})
119 
120 
121 def main():
122     parser = argparse.ArgumentParser(description=__doc__)
123     sub = parser.add_subparsers(dest='command', required=True)
124     h = sub.add_parser('hash'); h.add_argument('password')
125     b = sub.add_parser('build')
126     b.add_argument('--words', type=int, default=1)
127     b.add_argument('--height', type=int, default=500)
128     b.add_argument('--width', type=int, default=20)
129     b.add_argument('--seed', type=int, default=0)
130     b.add_argument('--output', default='tabela.json')
131     q = sub.add_parser('lookup'); q.add_argument('table'); q.add_argument('hash')
132     args = parser.parse_args()
133     if args.command == 'hash':
134         print(my_hash(args.password).hex())
135     elif args.command == 'build':
136         table = RainbowTable.build(Space(load_words(), args.words), args.height, args.width,
    args.seed)
137         table.save(args.output); print('Tabela salva em', args.output)
138     else:
139         print(RainbowTable.load(args.table).lookup(bytes.fromhex(args.hash)) or 'Não encontrada')
140 
141 
142 if __name__ == '__main__':
143     main()
```

Apêndice - test_rt.py

Código-fonte; linhas longas quebradas apenas para exibição.

```
001  import tempfile
002  import unittest
003  from pathlib import Path
004  from rt import Space, RainbowTable, my_hash, load_words
005
006
007  class Tests(unittest.TestCase):
008      def test_professor_vector(self):
009          self.assertEqual(my_hash('1234').hex(),
     '5042f15f93d4f75df9cdff09d062d607eaa0140a930b9023c43ca7cc58609b8f')
010
011      def test_official_list(self):
012          self.assertEqual(len(load_words()), 7776)
013
014      def test_encoding_bijection(self):
015          s = Space(['a', 'b', 'c'], 2)
016          self.assertEqual(len({s.password(i) for i in range(s.n)}), 9)
017          self.assertEqual(s.password(5), 'b c')
018
019      def test_reduction_range(self):
020          s = Space(['a', 'b', 'c'], 2)
021          for i in range(50):
022              self.assertIn(s.reduce(my_hash(str(i)), i), range(s.n))
023              self.assertEqual(s.reduce(my_hash(str(i)), i), s.reduce(my_hash(str(i)), i))
024
025      def test_exhaustive_lookup_matches_coverage(self):
026          # Oráculo independente: cobertura explícita em universos pequenos.
027          for k in (1, 2):
028              s = Space(['a', 'b', 'c', 'd'], k)
029              for width in (1, 2, 7):
030                  for seed in range(4):
031                      t = RainbowTable.build(s, 2, width, seed)
032                      covered = t.coverage()
033                      for i in range(s.n):
034                          found = t.lookup(my_hash(s.password(i)))
035                          self.assertEqual(found is not None, i in covered)
036                          if found is not None:
037                              self.assertEqual(found, s.password(i))
038
039      def test_duplicate_endpoints_and_roundtrip(self):
040          s = Space(['a', 'b', 'c', 'd'], 2)
041          t = RainbowTable.build(s, s.n, 15, 7)
042          self.assertTrue(any(len(v) > 1 for v in t.endpoints.values()))
043          with tempfile.TemporaryDirectory() as folder:
044              p = Path(folder) / 'table.json'; t.save(p)
045              restored = RainbowTable.load(p)
046              for i in range(s.n):
047                  self.assertEqual(restored.lookup(my_hash(s.password(i))), s.password(i))
048
049      def test_absent_and_invalid(self):
050          s = Space(['a', 'b']); t = RainbowTable.build(s, 1, 1)
051          self.assertIsNone(t.lookup(my_hash('fora-do-dicionario')))
052          with self.assertRaises(ValueError): RainbowTable.build(s, 3, 1)
053          with self.assertRaises(ValueError): t.lookup(b'bad')
054
055
056  if __name__ == '__main__':
057      unittest.main(verbosity=2)
```

Apêndice - experimentos.py

Código-fonte; linhas longas quebradas apenas para exibição.

```
001  """Experimentos sem dependências externas; resultados em JSON."""
002  import argparse
003  import hashlib
004  import json
005  import math
006  import os
007  import platform
008  import random
009  import statistics
010  import time
011  import tracemalloc
012  from pathlib import Path
013  from rt import Space, RainbowTable, load_words, my_hash, DEFAULT_LIST
014
015
016  def hardware():
017      cpu = platform.processor()
018      if Path('/proc/cpuinfo').exists():
019          cpu = next((line.split(':', 1)[1].strip() for line in
     Path('/proc/cpuinfo').read_text().splitlines() if line.startswith('model name')), cpu)
020      mem = None
021      if Path('/proc/meminfo').exists():
022          mem = int(Path('/proc/meminfo').read_text().splitlines()[0].split()[1]) * 1024
023      elif platform.system() == 'Windows':
024          import ctypes
025          class MemoryStatus(ctypes.Structure):
026              _fields_ = [('length', ctypes.c_ulong), ('load', ctypes.c_ulong)] + [(n,
     ctypes.c_ulonglong) for n in ['total', 'avail', 'page', 'availpage', 'virtual', 'availvirtual',
     'extended']]
027          status = MemoryStatus(); status.length = ctypes.sizeof(status)
028          if ctypes.windll.kernel32.GlobalMemoryStatusEx(ctypes.byref(status)):
029              mem = status.total
030      return {'cpu': cpu or 'não identificado', 'logical_cpus': os.cpu_count(),
031              'visible_ram_bytes': mem, 'platform': platform.platform(),
032              'python': platform.python_version(),
033              'cgroup': {name: (Path('/sys/fs/cgroup')/name).read_text().strip() for name in
     ('cpu.max','memory.max') if (Path('/sys/fs/cgroup')/name).exists()}, 'execution': 'sequencial, um
     processo; recursos visíveis podem ser do host'}
034
035
036  def wilson(success, count):
037      z = 1.96; p = success / count; d = 1 + z*z/count
038      center = (p + z*z/(2*count))/d
039      half = z*math.sqrt(p*(1-p)/count + z*z/(4*count*count))/d
040      return [center-half, center+half]
041
042
043  def run(trials, repeats, out):
044      out.mkdir(parents=True, exist_ok=True)
045      words = load_words(); results = []
046      configs = [(1,100,100), (1,200,50), (1,500,20),
047                 (2,2000,100), (2,4000,50), (2,10000,20)]
048      for k,m,t in configs:
049          s = Space(words,k)
050          for rep in range(repeats):
051              seed = 100 + rep
052              begin = time.perf_counter(); table = RainbowTable.build(s,m,t,seed)
053              build = time.perf_counter()-begin
054              path = out / 'ultima_tabela.json'; table.save(path)
055              disk = path.stat().st_size
056              # Medição de alocações separada para não contaminar os tempos.
057              tracemalloc.start()
058              measured = RainbowTable.build(s,m,t,seed)
059              _, peak = tracemalloc.get_traced_memory(); tracemalloc.stop()
060              del measured
061              covered = table.coverage()
062              # Mesmo conjunto por k/repetição em todas as configurações; independente dos inícios.
063              rng = random.Random(900000 + 1000*k + rep)
064              indices = [rng.randrange(s.n) for _ in range(trials)]
065              samples = []
066              for index in indices:
067                  target = my_hash(s.password(index))
068                  begin = time.perf_counter(); found = table.lookup(target)
069                  elapsed = time.perf_counter()-begin
070                  if (found is not None) != (index in covered):
```

Apêndice - experimentos.py

Código-fonte; linhas longas quebradas apenas para exibição.

```
                    raise AssertionError('Busca discordou da cobertura exata.')
                if found is not None and my_hash(found) != target:
                    raise AssertionError('Hash recuperado incorreto.')
                samples.append({'index':index,'found':found is not None,'seconds':elapsed})
            successes = sum(x['found'] for x in samples)
            row = {'k':k,'N':s.n,'height':m,'width':t,'seed':seed,
                   'build_s':build,'disk_bytes':disk,'python_peak_bytes':peak,
                   'distinct_covered':len(covered),'coverage':len(covered)/s.n,
                   'unique_ends':len(table.endpoints),'successes':successes,'trials':trials,
                   'success_rate':successes/trials,'wilson95':wilson(successes,trials),
                   'mean_lookup_ms':1000*statistics.mean(x['seconds'] for x in samples),
                   'median_lookup_ms':1000*statistics.median(x['seconds'] for x in samples),
                   'samples':samples}
            results.append(row)
            print(k,m,t,rep, f'sucesso={successes}/{trials} cobertura={row["coverage"]:.4%}',
flush=True)
            payload = {'hardware':hardware(),'trials':trials,'repeats':repeats,
                       'list_sha256':hashlib.sha256(DEFAULT_LIST.read_bytes()).hexdigest(),
                       'source':'https://theworld.com/~reinhold/diceware.wordlist.asc',
                       'results':results}

(out/'resultados.json').write_text(json.dumps(payload,indent=2,ensure_ascii=False),encoding='utf-8')
    path.unlink(missing_ok=True)


if __name__ == '__main__':
    p=argparse.ArgumentParser(); p.add_argument('--trials',type=int,default=200)
    p.add_argument('--repeats',type=int,default=3)
    p.add_argument('--output',type=Path,default=Path('resultados'))
    a=p.parse_args()
    if a.trials<1 or a.repeats<1: p.error('Valores devem ser positivos.')
    run(a.trials,a.repeats,a.output)
```